NOVAMENTE / ATLAS

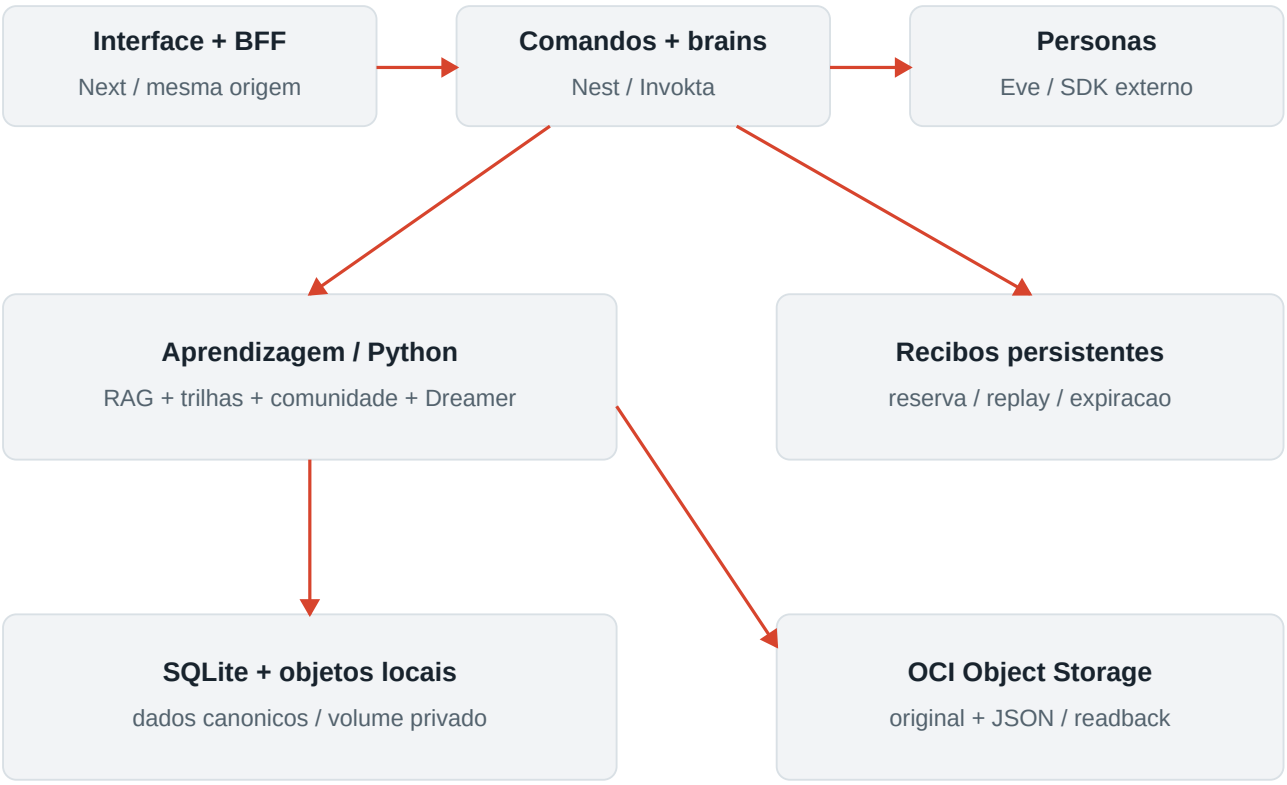
Atlas por dentro: guia técnico do código

Implementação 0.5, limites e próximos passos.
28/10/2026 · Material técnico da branch de revisão.

Baseline sincronizada: 3a58543
Branch: feat/andre-pos-mvp-quality-20261028
Fonte Markdown SHA256: 9a9d6178a93ab96f

Leia também SPEC, ENV, backlog, roadmap e parecer independente. Este PDF não comprova deploy, fornecedor real nem aprovação de produção.

Mapa dos módulos e processos



Next apresenta e protege a fronteira. Nest registra comandos e aplica capacidades. Python contém regras de estudo e memória. Eve executa as personas. O mapa resume dependências; as respostas retornam pelas mesmas fronteiras. Cofre e master ficam no servidor.

01. Como pensar o sistema

Imagine uma biblioteca com oficina de estudo. O navegador é o balcão; o Nest recebe pedidos e confere permissões; o Python organiza fontes, materiais e progresso; o Eve executa os tutores. A memória Dreamer aprende preferências pedagógicas consentidas. Os dados e efeitos são decididos no backend.

Um agente é uma persona com capacidades. Um método é uma abordagem como Feynman ou Socrático. Um formato é o produto, como quiz ou resumo. O provedor executa o modelo. Trocar o nome visual de Guia para Órion não muda essas regras.

02. Instalar, subir e identificar os processos

Requisitos: Node24 e Python3.12; dependências presas em lockfiles. Na raiz:

```
bash scripts/setup.sh
npm run dev
```

O setup cria `.env` local se ausente, token interno e master do cofre quando necessário, sem expor os valores. Não sobrescreva a master existente. O script de desenvolvimento inicia Python8000, Nest4000 e Next3000. Eve2000 é um processo separado para IA real. Sem chaves, a demonstração é extrativa e identifica isso.

`scripts/with-env.mjs` preserva variáveis injetadas e completa ausências do `.env`. `scripts/dev.sh` acompanha seus processos e encerra os grupos com Ctrl+C. O ambiente cloud não cria um preview público de loopback; os testes usam HTTP local e capturas de tela. Não abra as portas na Internet antes da autenticação.

03. Onde começar uma mudança

Tipo de mudança	Arquivo/pasta inicial
Regra de uma trilha	`services/cognition/atlas/trails/policy.py`
Persistir/anexar/concluir etapa	`services/cognition/atlas/trails/service.py`
Endpoint Python de trilhas	`services/cognition/atlas/trails/http.py`
Comando de trilha com geração	`apps/api/src/modules/trails.controller.ts`
Tela de trilhas	`apps/web/components/trails.tsx`
Quiz/XP	`services/cognition/atlas/attempts.py`, `progression.py`
Fontes/citações	`rag.py`, `learning.py`, `models.py`
Agentes	`packages/contracts/agents.json`, `apps/api/agent`
Ações permitidas	`apps/api/src/orchestration/actions.ts`
Execução/debate	`apps/api/src/orchestration/factory.ts`
Memória/autoaperfeiçoamento	`dreamer.py`, `rollout.py`, `replay.py`, `jobs.py`
Repetição de comandos	`apps/api/src/commands`
Cofre/provedor	`apps/api/src/connections`, `atlas/credentials.py`

O `main.ts`/`main.py` compõe a aplicação. Não colocar nova regra pedagógica dentro do bootstrap. `context.ts` liga implementações concretas; não é um módulo de domínio importável pela UI. Controllers validam e encaminham; serviços aplicam regras e transações; regras puras não conhecem HTTP, SDK ou tela.

04. Um pedido de material, do clique ao banco

1. `Atlas.generate` monta o objetivo/fontes/perfil/formato e chama `post`.
2. `lib/api.ts` atribui ID ao comando; retry manual pode reutilizá-lo.
3. A route Next chama `lib/server/proxy.ts`: origem, allowlist, tamanho e token interno. O browser não recebe esse token.
4. Nest autentica a chamada e `commandMiddleware` reserva o ID no SQLite.
5. `LearningController` valida Zod e chama `brains.respond`.
6. O brain confere fontes e política autorizada; `decide` escolhe método/RAG.
7. A capability Invokta valida entrada, saída e principal; chama Python.

8. `learning.generate` recupera evidências, solicita Eve quando configurado,

valida citações/judge, cria gabarito privado e persiste o pacote.

9. O comando salva a resposta no recibo antes de enviá-la; Next devolve JSON.

Chamadas externas podem falhar. A falha nunca autoriza inventar citação, score, recibo OCI ou trocar fornecedor silenciosamente. O usuário recebe um estado útil sem o debate privado ou pensamento interno dos modelos.

05. Entender os dados sem adivinhar

`Store` abre conexões SQLite curtas com commit/rollback. WAL permite leitores durante uma escrita. Não manter uma transação aberta enquanto espera o LLM. As consultas carregam tenant e recurso; o ID sozinho não autoriza leitura.

Tabela/grupo	O que guarda
documents, passages, passages_fts	Metadados, páginas, seções, representação e busca textual.
packages	JSON público, gabaritos privados e input usado.
xp, quiz_attempts	Recompensa única e resposta imutável por evento.
trails	Plano versionado por fonte/perfil/agente e estado de cada etapa.
groups, messages	Conversas locais; multiusuário depende de P02/P06.
consent, memory_revisions	Permissão e revisão que invalida escritores atrasados.
memory_events, agent_runs, dream_worlds, replay_cases	Experiências consentidas e dados separados de auditoria/replay.
policies, cycles, jobs	Políticas avaliadas, execuções e fila limitada.
commands.sqlite3 / receipts, epochs	Registro independente de comandos/replays e invalidação.

Migrações desta branch são aditivas com `CREATE TABLE IF NOT EXISTS`. Backup antes de trocar schema em produção. Depois de migration destrutiva, rollback de imagem pode não bastar; P14 define o plano de dados.

06. Idempotência: repetir sem duplicar o efeito

HTTP pode perder uma resposta mesmo depois de o servidor terminar. O cliente envia `Idempotency-Key` para identificar a intenção. A mesma chave no mesmo caminho com o mesmo JSON retorna a resposta original. JSON com ordem de campos diferente é igual; conteúdo diferente conflita.

```
# Execute somente contra seu workspace privado de desenvolvimento.
curl -X POST http://127.0.0.1:3000/api/atlas/groups \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: exemplo-grupo-comando-0001' \
  -d '{"name":"Redes OCI","topic":"Conferir fontes em grupo"}'
```

Repetir esse comando preserva o ID do grupo; mudar o tópico mantendo a chave retorna409. O ID não é uma senha. Se o processo cai após reservar, a chave fica pendente e não é liberada automaticamente. Conferir estado antes de criar nova intenção. Esta implementação não garante exactly-once em fornecedores externos.

O ledger não armazena requisições, só fingerprint e resposta admissível; nunca rastreia o corpo de configuração de chaves. Há TTL de conteúdo e quotas; chaves expiradas continuam bloqueadas. Clientes sem chave não têm proteção de replay neste incremento. P01 fecha essa compatibilidade e inclui outbox/reconciliação.

07. Quiz e progresso são comandos de domínio

`attempts.validate_answer` confere item/alternativa usando o gabarito privado. `register` abre `BEGIN IMMEDIATE`, consulta o evento e grava XP/recibo numa transação. `INSERT OR IGNORE` em XP protege o par pacote/questão contra duplo clique e concorrência. Um evento já usado com outra resposta retorna conflito.

O evento Dreamer ocorre depois da transação, sujeito a consentimento/revisão. Uma queda nesse intervalo pode perder a experiência; P01 resolve com outbox. Isso não reatribui XP. `progression.progress` calcula nível e intervalo; o browser só desenha a barra usando a projeção. Revelar flashcard não dá XP.

08. Como as trilhas funcionam

`trails.policy.steps` cria quatro etapas: compreender/tutorial, relembrar/cartões, praticar/quiz e sintetizar/resumo. Esse roteiro é identificado como local, independente da geração remota dos materiais. P04 adiciona planejamento por LLM.

O serviço fixa hashes das fontes e usa `selected_step` para bloquear saltos. Nest solicita o input autorizado ao Python, chama o mesmo brain de geração e anexa o material apenas se estiver pronto. `compatible` confere objetivo, formato, perfil, agente e referências. Material incompatível não avança a trilha.

Completar abre uma transação e confere estado novamente. Leituras são declaração do aluno; quiz precisa de acertos em XP para todas as questões. Conclusão repetida não soma etapas nem concede nova recompensa. O progresso persiste após reload.

09. Recuperação documental e economia de recursos

Upload começa por validação barata e hash. Um documento já conhecido no mesmo workspace retorna metadados sem repetir parser, embeddings ou gravação. Documento alheio nunca satisfaz essa consulta. PDF novo passa por subprocesso limitado.

Busca de seções carrega metadados e trechos, sem ler páginas integrais de todas as fontes. LM RAG lê o contexto integral dentro do orçamento; se exceder, troca para seções com explicação explícita, sem truncamento oculto. Modelos de embedding são opcionais na operação; hash lexical não é embedding semântico. Troca de endpoint/modelo exige reindexação com identidade consistente.

Uma CPU/RAM pequena pede poucos trabalhos pesados, textos/páginas limitados e inferência externa. Não adicionar Redis/Mongo/Qdrant/Trigger local apenas por parecer uma arquitetura profissional. Medir primeiro. Build e ferramentas de qualidade não pertencem à VPS operacional.

10. Brains, Invokta e Eve sem caixa preta

`createBrains` usa o catálogo e cria principal no servidor. `actions.ts` contém três engines: aprendizagem, pesquisa e memória. A access policy verifica tenant, role e ID coerentes; o modelo não escolhe um principal administrativo.

Factory assina tickets com propósito, evidências e orçamento. Eve recebe persona autenticada e SDK selecionado no servidor. Guia, Faísca, Prisma, Trama e Elo continuam nos IDs `nilo/lumi/prisma/fio/soma`. Os nomes mudaram, os dados não. Debates usam três a cinco turnos sequenciais com deadline global; a escolha de time e avaliação dependem de Jev real. Sem ele, não criar vencedor fictício.

`connections/eve-model.ts` resolve um modelo nativo em `step.started`, com teto de contexto. Não colocar key em prompt, estado durável ou tool input. Ferramentas têm schemas e capability; não dar shell/HTTP arbitrário ao tutor.

11. Dreamer: o que melhora e o que não pode mudar

Experiências reais consentidas produzem árvores/mundos avaliados. O Dreamer propõe política pedagógica limitada, simula histórico registrado e compara baseline/candidata em casos separados. Ganho de eficiência não compensa perda de suporte. Só promove com gates, versão e possibilidade de rollback.

O replay não reexecuta agente/judge para inventar ramos ausentes. Falta de cobertura é reportada. Revisão de memória invalida jobs/writers após revogação, rollback ou exclusão. Nenhuma política altera código, pesos, ferramentas ou permissões. Memória coletiva ainda exige implementação e consentimento separado.

12. Chaves, SDKs e variáveis

Leia `.env.example` e ENV.md. São fontes possíveis: env injetado, arquivo `_FILE` com prioridade ou chave manual cifrada. Next não recebe master nem lê o cofre. API/Python/Eve usam volume privado e UID10001. Backup da master deve ser separado; perder a master perde a capacidade de ler chaves cifradas.

O painel informa configuração, não validação de geração. Um teste de catálogo não prova ferramentas/JSON. Login ChatGPT nativo Eve é somente dev local explicitamente habilitado; não é login OAuth do aluno ou API de assinatura em produção. Nunca reutilizar autenticação da ferramenta que desenvolve o projeto.

13. Testar, medir e interpretar o resultado

```
npm test
npm run check:env
npm run typecheck
npm run build
npm run quality:setup
npm run quality
npx playwright install chromium
npm run test:e2e
npm run test:runtime
```

No cloud, um Chromium já instalado pode ser selecionado com `ATLAS_E2E_CHROMIUM=/usr/bin/chromium`; não desabilitar TLS nem contornar allowlist para instalar browsers. O E2E exige build prévio e executa Next standalone/Nest compilado, sem watchers. Cria dados isolados e desliga credenciais de provedor. `ATLAS_E2E_REUSE=1` só deve reutilizar servidor de teste conhecido.

Unidade verifica regra isolada; integração verifica fronteiras; E2E verifica a jornada real. Testes controlados não provam qualidade de LLM/OCI. O relatório está em `artifacts/quality/report.json`; mede novos limites e regressões, registra dívida legada e aplica também o ratchet à UI. Métricas não são aprovação humana. Não reduzir política nem remover teste para obter verde.

14. Exemplo de contribuição sem quebrar o sistema

Para acrescentar uma etapa à trilha: primeiro escreva exemplos de desbloqueio, conclusão e material incompatível; depois altere a política e schema versionado. Atualize serviço, contratos TS, apresentação e migração de planos existentes. Rode unidade/HTTP/E2E da trilha e análise estrutural. Documente comportamento e limites no PR; não copie a lógica de desbloqueio para React.

Use a branch atual e commits pequenos. `scripts/push-branch-both.sh` recusa main/master, exige árvore limpa e verifica SHA nos dois remotes. O script antigo `push-both.sh` publica main e

não deve ser usado nesta entrega. Nenhum merge é automático. Depois de revisão humana e CI real, aplicar o plano de deploy.

15. Diagnóstico por sintoma

Sintoma	Inspecionar primeiro	Conduta
Workspace sem carregar	Bootstrap/BFF e logs locais sem segredos	Conferir token/ports/env loader; não publicar para “testar”.
Material extrativo	Capabilities e geração configurada	Esse modo é explícito; configurar Eve/modelo para IA real.
409 ao repetir comando	Estado do recurso e recibo	Não gerar IDs em loop para contornar resultado incerto.
429	Fila e trabalho ativo	Aguardar; não abrir mais workers sem orçamento global.
Etapa não avança	Material, pré-requisito e acertos no servidor	Não editar unlocked/XP no browser.
Embedding mudou	Identidade de endpoint/modelo	Reindexar; nunca misturar dimensões/índices.
Master ausente	Secret manager/backup separado	Restaurar a master correta; não gerar outra por cima.
OCI não verified	Upload/readback/IAM/quota	Falha não fecha o MVP; seguir U06/P03.

16. O que ainda exige implementação ou prova

O roadmap lista quinze pacotes e ações U01–U16. Autenticação, autorização por usuário e cofre individual bloqueiam uso público. Fornecedores, vetores e OCI precisam de aceite real. Trilhas LLM, colaboração real/coletiva, Obsidian, anotações, OCR e polimento ampliado têm contratos e plano, mas não estão concluídos nesta branch. RAM de produção, restauração e eficácia com alunos precisam de medição própria. O projeto continua evolutivo e revisável.